# Enhancing spatial safety: fixing thousands of -Wflex-array-member-not-at-end warnings

Gustavo A. R. Silva
gustavo@embeddedor.com
fosstodon.org/@gustavoars

Supported by
The Linux Foundation & Alpha-Omega

Linux Plumbers Conference
September 19, 2024
Vienna, Austria

# Agenda

- **Introduction**
  - C99 flexible-array members (FAMs)
  - The new -Wflex-array-member-not-at-end compiler option
- **The challenge of -Wflex-array-member-not-at-end**
  - What’s wrong with FAMs in the middle?
  - How did we get here? - A brief history
  - Fixing thousands of -Wfamnae warnings
- **Conclusions**

# Quick review of C99 flexible-array members

- The last member of a struct.

```
struct flex {
    ...

    struct foo fam[];
};
```

# Extended review of C99 flexible-array members

- The last member of a struct.
- The flex struct usually contains a ***counter*** member.

```
struct flex {
    ...
    size_t  count;
    struct foo fam[];
};
```

# Extended review of C99 flexible-array members

- The last member of a struct.
- The flex struct usually contains a ***counter*** member.
- Run-time bounds-checking coverage.

```
struct flex {
    ...
    size_t count;
    struct foo fam[] __counted_by(count);
};
```

# The new -Wflex-array-member-not-at-end

- Developed by Qing Zhao last year (2023)
- Released in GCC 14

# The new -Wflex-array-member-not-at-end

- Flex-array member in the middle of a composite struct.

```
struct flex {
    ...
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct composite {
    ...

    struct flex middle;


    ...
};
```

# The new -Wflex-array-member-not-at-end

- Flex-array member in the middle of a composite struct.

```
struct flex {
    ...
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct composite {
    ...

    struct flex middle; /* -Wfamnae warning */

    ...
};
```

# The challenge of enabling

# **-Wflex-array-member-not-at-end**

# What’s wrong with FAMs in the middle?

– Flex struct in a composite struct is an extension.

# What’s wrong with FAMs in the middle?

- Flex struct in a composite struct is an extension.
  - the last member

```
struct composite {
   ...

   struct flex last;
};
```

# What’s wrong with FAMs in the middle?

- Flex struct in a composite struct is an extension.
  - the last member
  - **not the last member**

```c
struct composite {
    ...

    struct flex last;
};
```

```c
struct composite {
    ...

    struct flex middle;

    ...
};
```

# What’s wrong with FAMs in the middle?

- Flex struct in a composite struct is an extension.
  - the last member
  - **not the last member – This is deprecated now.**

```
struct composite {
   ...

   struct flex last;
};
```

```
struct composite {
   ...

   struct flex middle;

   ...
};
```

# What’s wrong with FAMs in the middle?

- Flex struct in a composite struct is an extension.
  - the last member
  - **not the last member – This is deprecated now.**

```
struct composite {
   ...

   struct flex last;
};
```

```
struct composite {
   ...

   struct flex middle;

   ...
};
```

# What’s wrong with FAMs in the middle?

- “Compilers do not handle such a case consistently. Any code relying on this case should be modified to **ensure that flexible array members only end up at the ends of structures.**” -GCC Docs.

```
struct composite {
    ...
    struct flex middle;

    ...
};
```

# How did we get here? - A brief history

- Flexible-Array Transformations - [1] & [0] to C99 [ ]
  - It took us 5 years (2019 – 2024)

# How did we get here? - A brief history

- Flexible-Array Transformations - [1] & [0] to C99 [ ]
  - It took us 5 years (2019 – 2024)
- [1], [0], [ ] & [N] trailing arrays & fortified memcpy()
  - Fixed __builtin_object_size()
  - Fixed __builtin_dynamic_object_size()

# How did we get here? - A brief history

- Flexible-Array Transformations - [1] & [0] to C99 [ ]
  - It took us 5 years (2019 – 2024)
- [1], [0], [ ] & [N] trailing arrays & fortified memcpy()
  - Fixed __builtin_object_size()
  - Fixed __builtin_dynamic_object_size()
  - -fstrict-flex-arrays[=n] – Clang 16 & GCC 13
  - -fstrict-flex-arrays=3 enabled in Linux 6.5
    - Only C99 FAMs are considered flex arrays or VLOs.

# How did we get here? - A brief history

- The *counted_by* attribute – Clang 18 & GCC 15

# How did we get here? - A brief history

- The *counted_by* attribute – Clang 18 & GCC 15
  - Use __builtin_dynamic_object_size() in fortified memcpy().
  - *counted_by* annotations in progress.

# How did we get here? - A brief history

- The *counted_by* attribute – Clang 18 & GCC 15
  - Use __builtin_dynamic_object_size() in fortified memcpy().
  - *counted_by* annotations in progress.
  - Ideally, every FAM should be annotated.

# Fixing thousands of -Wfamnae warnings in Linux

- A bit more than 60,000 warnings in total

```
struct flex {
    ...
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct composite {
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
};
```

# Fixing thousands of -Wfamnae warnings in Linux

- A bit more than 60,000 warnings in total – 650 unique ones.

```
struct flex {
    ...
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct composite {
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
};
```

# Fixing thousands of -Wfamnae warnings in Linux

- A bit more than 60,000 warnings in total – 650 unique ones.
- Some patterns emerged.

```
struct flex {
    ...
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct composite {
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
};
```

# -Wflex-array-member-not-at-end

Case 1: **FAMs not used at all.**

# -Wflex-array-member-not-at-end

Case 1: **FAMs not used at all**.

- f4b09b29f8b4 (“wifi: ti: Avoid a hundred -Wflex-array-member-not-at-end warnings”)

```
struct wl1251_cmd_header {
   u16 id;
   u16 status;
-  /* payload */
-  u8 data[];
 } __packed;
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

# -Wflex-array-member-not-at-end

## Case 2: **FAMs never accessed through the composite struct**.

```
struct flex {
    int a;
    int b;
    size_t  count;
    struct foo fam[];
};

struct composite {
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
} *p;
...


do_something_with(p->middle.a, p->middle.b);
```

# -Wflex-array-member-not-at-end

## Case 2: **FAMs never accessed through the composite struct**.

```
struct flex {
    int a;
    int b;
    size_t  count;
    struct foo fam[];
};

struct composite {
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
} *p;
...

/* We may access the rest of the members in struct flex */
do_something_with(p->middle.a, p->middle.b);
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- What can we do about it?

```
struct flex {
    int a; int b;
    size_t count;
    struct foo fam[];
};

struct composite {
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

```
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr {
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

– Two separate structs: original struct & header struct

```
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr {
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Two separate structs: original struct & header struct
- New header struct named after original flex struct.

```
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr {
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Two separate structs: original struct & header struct
- New header struct named after original flex struct.

```
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr { /* All members in struct flex except the FAM */
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

- Case 2: **FAMs never accessed through the composite struct**.
  - Two separate structs: original struct & header struct

```
struct composite { /* BEFORE */
    ...
    struct flex middle; /* -Wfamnae warning :( */
    ...
};

struct composite { /* AFTER */
    ...
    struct flex_hdr middle;
    ...
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Two separate structs: original struct & header struct

```
struct composite { /* BEFORE */
    ...
    struct flex middle; /* -Wfamnae warning :( */
    ...
};

struct composite { /* AFTER */
    ...
    struct flex_hdr middle;
    ...
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Two separate structs: original struct & header struct

```
struct composite { /* BEFORE */
    ...
    struct flex middle; /* -Wfamnae warning :( */
    ...
};

struct composite { /* AFTER */
    ...
    struct flex_hdr middle; /* Life’s beautiful! ^.^ */
    ...
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

– What’s the problem with this?

```c
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr { /* All members in struct flex except the FAM */
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Duplicate code.

```
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr { /* All members in struct flex except the FAM */
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Duplicate code.

```
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr { /* All members in struct flex except the FAM */
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Duplicate code.
- Maintain two independent but basically identical structs.

```
struct flex { /* original struct */
    int a; int b;
    size_t count;
    struct foo fam[] __counted_by(count);
};

struct flex_hdr { /* All members in struct flex except the FAM */
    int a; int b;
    size_t count;
};
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Use struct_group_tagged()/__struct_group()

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Use struct_group_tagged()/__struct_group()

```
struct flex { /* BEFORE */


    int a; int b;
    size_t  count;

    struct foo fam[] __counted_by(count);
};

struct composite { /* BEFORE */
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
} *p;
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Use struct_group_tagged()/__struct_group()

```
struct flex { /* AFTER */
    /* New members must be added within the struct_group() macro below. */
    struct_group_tagged(flex_hdr, hdr,
        int a; int b;
        size_t  count;
    );
    struct foo fam[] __counted_by(count);
};

struct composite { /* BEFORE */
    ...
    struct flex middle; /* -Wfamnae warning */
    ...
} *p;
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Use struct_group_tagged()/__struct_group()

```
struct flex { /* AFTER */
    /* New members must be added within the struct_group() macro below. */
    struct_group_tagged(flex_hdr, hdr,
        int a; int b;
        size_t  count;
    );
    struct foo fam[] __counted_by(count);
};

struct composite { /* AFTER */
    ...
    struct flex_hdr middle; /* FAM is gone! ^.^ */
    ...
} *p;
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

- Use struct_group_tagged()/__struct_group()

```
struct flex { /* AFTER */
    /* New members must be added within the struct_group() macro below. */
    struct_group_tagged(flex_hdr, hdr,
        int a; int b;
        size_t  count;
    );
    struct foo fam[] __counted_by(count);
};

struct composite { /* AFTER */
    ...
    struct flex_hdr middle; /* FAM is gone! ^.^ */
    ...
} *p;
```

```
p->middle.a
p->middle.b
p->middle.count

p->middle.hdr.a
p->middle.hdr.b
p->middle.hdr.count
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

– 5c4250092fad (“wifi: mwl8k: Avoid -Wflex-array-…”)

```
 struct mwl8k_cmd_pkt {
-   __le16  code;
-   __le16  length;
-   __u8    seq_num;
-   __u8    macid;
-   __le16  result;
+   __struct_group(mwl8k_cmd_pkt_hdr, hdr, __packed,
+       __le16  code;
+       __le16  length;
+       __u8    seq_num;
+       __u8    macid;
+       __le16  result;
+   );
    char payload[];
 } __packed;
```

# -Wflex-array-member-not-at-end

Case 2: **FAMs never accessed through the composite struct**.

– 5c4250092fad (“wifi: mwl8k: Avoid -Wflex-array-…”)

```
 struct mwl8k_cmd_pkt {
-	__le16	code;
-	__le16	length;
-	__u8	seq_num;
-	__u8	macid;
-	__le16	result;
+	__struct_group(mwl8k_cmd_pkt_hdr, hdr, __packed,
+		__le16	code;
+		__le16	length;
+		__u8	seq_num;
+		__u8	macid;
+		__le16	result;
+	);
	char payload[];
 } __packed;
```

# -Wflex-array-member-not-at-end

- 5c4250092fad (“wifi: mwl8k: Avoid -Wflex-array-…”)
- Replace *mwl8k_cmd_pkt* with *mwl8k_cmd_pkt_hdr*

```
 struct mwl8k_cmd_rf_antenna {
-  struct mwl8k_cmd_pkt header;
+  struct mwl8k_cmd_pkt_hdr header;
   __le16 antenna;
   __le16 mode;
 } __packed;
```

# -Wflex-array-member-not-at-end

- 5c4250092fad (“wifi: mwl8k: Avoid -Wflex-array-…”)
- Replace *mwl8k_cmd_pkt* with *mwl8k_cmd_pkt_hdr*

```
 struct mwl8k_cmd_rf_antenna {
-  struct mwl8k_cmd_pkt header;
+  struct mwl8k_cmd_pkt_hdr header;
   __le16 antenna;
   __le16 mode;
 } __packed;
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
struct flex_struct {
    ...
    size_t  count;
    struct foo flex_array[] __counted_by(count);
};

struct composite_struct {
    ...

    struct flex_struct flex_in_the_middle;
    struct foo fixed_array[MAX_LENGTH];
    ...
} __packed;
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
struct flex_struct {
    ...
    size_t  count;
    struct foo flex_array[] __counted_by(count);
};

struct composite_struct {
    ...

    struct flex_struct flex_in_the_middle;
    struct foo fixed_array[MAX_LENGTH];
    ...
} __packed;
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
struct flex_struct {
    ...
    size_t  count;
    struct foo flex_array[] __counted_by(count);
};

struct composite_struct {
    ...

    struct flex_struct flex_in_the_middle;
    struct foo fixed_array[MAX_LENGTH];
    ...
} __packed;
```

- `flex_array` and `fixed_array` share the same address in memory - in the best scenario.
- Both form an implicit union.

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- However, **FAM digest** is the one accessed at run-time.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

– However, **FAM digest** is the one accessed at run-time.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- Use **container_of()** to get a pointer to the flex struct.
- Access FAM through that pointer.

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- Use **container_of()** to get a pointer to the flex struct.
- Access FAM through that pointer.

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                             struct ima_digest_data, hdr);

... hash_hdr is now a pointer to flex struct ima_digest_data

        /* read data from the FAM digest */
-       memcpy(digest_hash, hash.hdr.digest, digest_hash_len);
+       memcpy(digest_hash, hash_hdr->digest, digest_hash_len);
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- Use **container_of()** to get a pointer to the flex struct.
- Access FAM through that pointer.

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                           struct ima_digest_data, hdr);

... hash_hdr is now a pointer to flex struct ima_digest_data

        /* read data from the FAM digest */
-       memcpy(digest_hash, hash.hdr.digest, digest_hash_len);
+       memcpy(digest_hash, hash_hdr->digest, digest_hash_len);
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- Use **container_of()** to get a pointer to the flex struct.
- Access FAM through that pointer.

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                           struct ima_digest_data, hdr);

... hash_hdr is now a pointer to flex struct ima_digest_data

        /* read data from the FAM digest */
-       memcpy(digest_hash, hash.hdr.digest, digest_hash_len);
+       memcpy(digest_hash, hash_hdr->digest, digest_hash_len);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- Use **container_of()** to get a pointer to the flex struct.
- Access FAM through that pointer.

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                         struct ima_digest_data, hdr);

... hash_hdr is now a pointer to flex struct ima_digest_data

        /* read data from the FAM digest */
-       memcpy(digest_hash, hash.hdr.digest, digest_hash_len);
+       memcpy(digest_hash, hash_hdr->digest, digest_hash_len);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                                  struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                                 struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                              struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                              struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                                   struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                                  struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                                struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

## Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

```
 struct ima_digest_data { /* flexible struct */
+   /* New members must be added within the __struct_group() macro below. */
+   __struct_group(ima_digest_data_hdr, hdr, __packed,
        u8 algo;
        u8 length;
        ...
+   );
    u8 digest[];
 } __packed;
```

```
/* implicit union: FAM & fixed-size array*/
 struct ima_max_digest_data {
-   struct ima_digest_data hdr;
+   struct ima_digest_data_hdr hdr;
    u8 digest[HASH_MAX_DIGESTSIZE];
 } __packed;
```

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                                   struct ima_digest_data, hdr);
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- Use **container_of()** to get a pointer to the flex struct.
- Access FAM through that pointer.
- 38aa3f5ac6d2 (“integrity: Avoid -Wflex-array-member...”)

```
    struct ima_max_digest_data hash; /* struct with implicit union */
+   struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+                                      struct ima_digest_data, hdr);

... hash_hdr is now a pointer to flex struct ima_digest_data

        /* read data from the FAM digest */
-       memcpy(digest_hash, hash.hdr.digest, digest_hash_len);
+       memcpy(digest_hash, hash_hdr->digest, digest_hash_len);
```

# -Wflex-array-member-not-at-end

Case 3: **Implicit unions** between FAMs and fixed-size arrays of the same element type.

- Use **container_of()** to get a pointer to the flex struct.
- Access FAM through that pointer.
- 38aa3f5ac6d2 (“integrity: Avoid -Wflex-array-member...”)

```
	struct ima_max_digest_data hash; /* struct with implicit union */
+	struct ima_digest_data *hash_hdr = container_of(&hash.hdr,
+					struct ima_digest_data, hdr);

... hash_hdr is now a pointer to flex struct ima_digest_data

		/* read data from the FAM digest */
-		memcpy(digest_hash, hash.hdr.digest, digest_hash_len);
+		memcpy(digest_hash, hash_hdr->digest, digest_hash_len);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
struct flex_struct {
    ...
    size_t  count;
    struct foo flex_array[] __counted_by(count);
};


int some_function(...) /* on-stack -Wfamnae warning */
{
    struct {
        struct flex_struct flex;
        struct foo fixed_array[10];
    } obj = ...
    ...
}
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
struct flex_struct {
    ...
    size_t  count;
    struct foo flex_array[] __counted_by(count);
};


int some_function(...) /* on-stack -Wfamnae warning */
{
    struct {
        struct flex_struct flex;
        struct foo fixed_array[10];
    } obj = ...
    ...
}
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
struct fun_admin_bind_req {
    struct fun_admin_req_common common;
    struct fun_admin_bind_entry entry[];
};

int fun_bind(...) /* on-stack -Wfamnae warning */
{
    struct {
        struct fun_admin_bind_req req;
        struct fun_admin_bind_entry entry[2];
    } cmd = ...
    ...
}
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
struct fun_admin_bind_req {
    struct fun_admin_req_common common;
    struct fun_admin_bind_entry entry[];
};

int fun_bind(...) /* on-stack -Wfamnae warning */
{
    struct {
        struct fun_admin_bind_req req;
        struct fun_admin_bind_entry entry[2];
    } cmd = ...
    ...
}
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
struct fun_admin_bind_req {
    struct fun_admin_req_common common;
    struct fun_admin_bind_entry entry[];     /* flex-array member */
};

int fun_bind(...) /* on-stack -Wfamnae warning */
{
    struct {
        struct fun_admin_bind_req req;
        struct fun_admin_bind_entry entry[2]; /* fixed-size array */
    } cmd = ...
    ...
}
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
-	struct {
-		struct fun_admin_bind_req req;
-		struct fun_admin_bind_entry entry[2];
-	} cmd = {
-		.req.common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
-						   sizeof(cmd)),
-		.entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0),
-		.entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1),
-	};
+	DEFINE_RAW_FLEX(struct fun_admin_bind_req, cmd, entry, 2);
+
+	cmd->common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
+					   __struct_size(cmd));
+	cmd->entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0);
+	cmd->entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
-   struct {
-       struct fun_admin_bind_req req;
-       struct fun_admin_bind_entry entry[2];
-   } cmd = {
-       .req.common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
-                                    sizeof(cmd)),
-       .entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0),
-       .entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1),
-   };
+   DEFINE_RAW_FLEX(struct fun_admin_bind_req, cmd, entry, 2);
+
+   cmd->common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
+                                 __struct_size(cmd));
+   cmd->entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0);
+   cmd->entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
-	struct {
-		struct fun_admin_bind_req req;
-		struct fun_admin_bind_entry entry[2];
-	} cmd = {
-		.req.common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
-							 sizeof(cmd)),
-		.entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0),
-		.entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1),
-	};
+	DEFINE_RAW_FLEX(struct fun_admin_bind_req, cmd, entry, 2);
+
+	cmd->common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
+						 __struct_size(cmd));
+	cmd->entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0);
+	cmd->entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
-   struct {
-       struct fun_admin_bind_req req;
-       struct fun_admin_bind_entry entry[2];
-   } cmd = {
-       .req.common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
-                                               sizeof(cmd)),
-       .entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0),
-       .entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1),
-   };
+   DEFINE_RAW_FLEX(struct fun_admin_bind_req, cmd, entry, 2);
+
+   cmd->common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
+                                           __struct_size(cmd));
+   cmd->entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0);
+   cmd->entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
-	struct {
-		struct fun_admin_bind_req req;
-		struct fun_admin_bind_entry entry[2];
-	} cmd = {
-		.req.common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
-						       sizeof(cmd)),
-		.entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0),
-		.entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1),
-	};
+	DEFINE_RAW_FLEX(struct fun_admin_bind_req, cmd, entry, 2);
+
+	cmd->common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
+						 __struct_size(cmd));
+	cmd->entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0);
+	cmd->entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
-	struct {
-		struct fun_admin_bind_req req;
-		struct fun_admin_bind_entry entry[2];
-	} cmd = {
-		.req.common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
-							sizeof(cmd)),
-		.entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0),
-		.entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1),
-	};
+	DEFINE_RAW_FLEX(struct fun_admin_bind_req, cmd, entry, 2);
+
+	cmd->common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
+						__struct_size(cmd));
+	cmd->entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0);
+	cmd->entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

```
-	struct {
-		struct fun_admin_bind_req req;
-		struct fun_admin_bind_entry entry[2];
-	} cmd = {
-		.req.common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
-						sizeof(cmd)),
-		.entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0),
-		.entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1),
-	};
+	DEFINE_RAW_FLEX(struct fun_admin_bind_req, cmd, entry, 2);
+
+	cmd->common = FUN_ADMIN_REQ_COMMON_INIT2(FUN_ADMIN_OP_BIND,
+					__struct_size(cmd));
+	cmd->entry[0] = FUN_ADMIN_BIND_ENTRY_INIT(type0, id0);
+	cmd->entry[1] = FUN_ADMIN_BIND_ENTRY_INIT(type1, id1);
```

# -Wflex-array-member-not-at-end

Case 4: **Implicit unions** between FAMs and fixed-size arrays of the same element type – **on stack**.

- We use **DECLARE_FLEX()** and **DECLARE_RAW_FLEX()** helpers.
- Some examples:
  - 6c85a13b133f (“platform/chrome: cros_ec_proto:…”)
  - 4d69c58ef2e4 (“fsnotify: Avoid -Wflex-array-mem…”)
  - 215c4704208b (“Bluetooth: L2CAP: Avoid -Wflex-...”)

# Conclusions

- Clear strategy to enable **-Wflex-array-member-not-at-end** in mainline, soon.
- A couple dozen patches are already in mainline.
- Down to ~300 unique warnings.
- ~30% of total warnings addressed in the last months.

# Thank you, Vienna!

Gustavo A. R. Silva
gustavoars@kernel.org
fosstodon.org/@gustavoars

By @shidokou